Smart University Management System (SUMS) Complete Project Documentation + Deployment Guide

An enterprise-grade AI-enabled university management platform built using Django and Python technologies. This project demonstrates backend engineering, scalable architecture, security engineering, automation, DevOps, and AI integration.

1. Project Overview

SUMS is a multi-role academic management platform designed for students, teachers, and administrators. The platform focuses on security, automation, scalability, and intelligent academic analytics.

---

2. Main Goals

• Centralize university operations
• Implement secure RBAC architecture
• Integrate AI-powered educational tools
• Provide scalable backend infrastructure
• Demonstrate production-ready Django engineering

---

3. Tech Stack

Backend: Django, Django REST Framework
Database: PostgreSQL
Cache: Redis
Task Queue: Celery
Deployment: Docker, Gunicorn, Nginx
Authentication: JWT
CI/CD: GitHub Actions
AI: Scikit-learn, NLP models

---

4. Core Features

Student Features:
• Course registration
• Assignment upload
• Attendance tracking
• Routine management
• CGPA analytics

Teacher Features:
• Grading system
• AI assignment summary
• Exam analytics
• Plagiarism detection

Admin Features:
• Role-based access control
• Audit logs
• Notice automation
• User management

---

5. Unique Features

• AI weak student prediction
• Login anomaly detection
• Academic chatbot assistant
• Automated schedule conflict resolver
• AI-powered plagiarism analysis
• Internal academic reputation score

---

6. Security Features

• JWT Authentication
• RBAC Authorization
• Rate Limiting
• Secure Password Hashing
• SQL Injection Protection
• CSRF Protection
• Login anomaly detection
• Audit logging

---

7. Suggested Database Tables

• users
• students
• teachers
• departments
• courses
• enrollments
• assignments
• attendance_records
• notices
• exam_results
• audit_logs

---

8. Suggested Folder Structure

project_root/
■■■ accounts/
■■■ students/
■■■ teachers/
■■■ courses/
■■■ analytics/
■■■ ai_engine/
■■■ security/
■■■ notifications/
■■■ api/
■■■ config/
■■■ docker/

---

9. Development Phases

Phase 1: Authentication & RBAC
Phase 2: Course & Assignment Management
Phase 3: Attendance & Analytics
Phase 4: AI Integration
Phase 5: Security Hardening
Phase 6: Deployment & CI/CD

---

10. Deployment Architecture

GitHub Repository
↓
Render Web Service (Django + Gunicorn)
↓
Neon PostgreSQL
↓
Upstash Redis
↓
Render Background Worker (Celery)

---

11. Free Deployment Services

Backend Hosting: Render
PostgreSQL: Neon
Redis: Upstash Redis
Media Storage: Cloudinary
CI/CD: GitHub Actions

---

12. Step-by-Step Deployment Guide

Step 1 — Install Production Packages

Install Required Packages

```
pip install gunicorn whitenoise psycopg2-binary celery redis dj-database-url
```

Generate requirements.txt

```
pip freeze > requirements.txt
```

Create Procfile

```
web: gunicorn config.wsgi
worker: celery -A config worker -l info
```

Configure Allowed Hosts

```
ALLOWED_HOSTS = ["*"]
```

Configure Static Files

```
MIDDLEWARE = [
    "whitenoise.middleware.WhiteNoiseMiddleware",
]

STATIC_ROOT = BASE_DIR / "staticfiles"
```

Configure PostgreSQL

```
import dj_database_url

DATABASES = {
    "default": dj_database_url.config(
        default=os.environ.get("DATABASE_URL")
    )
}
```

Configure Celery Redis Broker

```
CELERY_BROKER_URL = os.environ.get("REDIS_URL")
```

Dockerfile

```
FROM python:3.12

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt
```

```
COPY . .

CMD gunicorn config.wsgi:application --bind 0.0.0.0:8000
```

Push Project to GitHub

```
git init
git add .
git commit -m "initial commit"
git branch -M main
git remote add origin YOUR_REPO
git push -u origin main
```

Render Build Command

```
pip install -r requirements.txt
```

Render Start Command

```
gunicorn config.wsgi:application
```

Environment Variables

```
DATABASE_URL=xxxx
REDIS_URL=xxxx
SECRET_KEY=xxxx
DEBUG=False
```

Run Database Migrations

```
python manage.py migrate
```

Collect Static Files

```
python manage.py collectstatic --noinput
```

Celery Worker Command

```
celery -A config worker -l info
```

docker-compose.yml

```
version: '3'

services:
  web:
    build: .
    command: gunicorn config.wsgi:application --bind 0.0.0.0:8000
    ports:
      - "8000:8000"

  redis:
    image: redis:alpine

  celery:
```

```
    build: .
    command: celery -A config worker -l info
```

13. Final Recommendation

Final Recommendation:

This project is highly suitable for:
• Backend Engineering Portfolio
• Django/Python Developer Resume
• Security Engineering Projects
• Research/Graduate Applications
• Enterprise Software Demonstration

Recommended Final Stack:
Django + DRF + PostgreSQL + Redis + Celery + Docker + JWT + Render Deployment